MEMORIA TÉCNICA DE REFACTORIZACIÓN

Informe Factorización

sobre

Proyecto Market

UPV/EHU, Universidad del País Vasco
Facultad de Informática
Grado en Ingeniería Informática
Ingeniería del Software II
Curso 2026 - 2027

AUTORES

Rafael Concejo Fernández
Kailai Ye Yin

FECHA Y REPOSITORIO

18/10/2026

https://github.com/Kailai123123/Proyecto_ISII

Índice

- createSale 3
- 1. Keep unit interfaces small (Mantener interfaces de unidades pequeñas - Capítulo 5) 5
- 2. Write short units of code (Escribir unidades de código cortas - Capítulo 2) 6
- 3. Write simple units of code (Escribir unidades de código simples - Capítulo 3) 7
- DataAccess: getProducts⇒ 8
- 4. Write code once / Avoid Duplication (Escribir el código una sola vez - Capítulo 4) 8
- addSale 9
- 1. Keep unit interfaces small (Mantener interfaces de unidades pequeñas - Capítulo 5) 9
- aceptarOferta 10
- 2. Write short units of code (Escribir unidades de código cortas - Capítulo 2) 11
- getProductos Comprados 12
- 3. Write simple units of code (Escribir unidades de código simples - Capítulo 3) 13
- "SELECT FROM Offer o JOIN o.sale s WHERE s.title LIKE ?1" 14
- 4. Write code once / Avoid Duplication (Escribir el código una sola vez - Capítulo 4) 14
- Referencia e Información 15
- Apartado uso de IA 16

createSale

CÓDIGO INICIAL

DATAACCESS.JAVA (INICIAL)

```
public Sale createSale(String title, String description, int status, float price,
                       Date pubDate, String sellerEmail, File file)
        throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
    try {
        String etiquetas = "Etiquetas";
        System.out.println(">> DataAccess: createProduct => title=" + title + " seller=" + sellerEmail);

        if (pubDate.before(UtilDate.trim(new Date()))) {
            throw new MustBeLaterThanTodayException(
                ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
        }
        if (file == null) {
            throw new FileNotUploadedException(
                ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorFileNotUploadedException"));
        }

        db.getTransaction().begin();
        Seller seller = db.find(Seller.class, sellerEmail);

        if (seller.doesSaleExist(title)) {
            db.getTransaction().commit();
            throw new SaleAlreadyExistException(
                ResourceBundle.getBundle(etiquetas).getString("DataAccess.SaleAlreadyExist"));
        }

        Sale sale = seller.addSale(title, description, status, price, pubDate, file);
        db.persist(seller);
        db.getTransaction().commit();

        System.out.println("sale stored " + sale + " " + seller);
        System.out.println("hasta aqui");
        return sale;
    } catch (NullPointerException e) {
        e.printStackTrace();
        db.getTransaction().commit();
        return null;
    }
}
```

Figura 1: Código inicial de createSale

CÓDIGO FINAL

DATAACCESS.JAVA (REFACTORIZADO)

```java
public Sale createSale(String sellerEmail, SaleDetails details)
        throws FileNotUploadedException, MustBeLaterThanTodayException, SaleAlreadyExistException {
    try {
        validateSaleInputs(details);
        db.getTransaction().begin();
        Seller seller = db.find(Seller.class, sellerEmail);
        checkSaleExistence(seller, details.getTitle());

        Sale sale = seller.addSale(details.getTitle(), details.getDesc(), details.getStatus(),
                                   details.getPrice(), details.getDate(), details.getFile());
        db.persist(seller);
        db.getTransaction().commit();

        Logger.info("Sale stored: " + sale.getTitle());
        return sale;
    } catch (NullPointerException e) {
        Logger.log(Level.INFO, "Error en createSale", e);
        db.getTransaction().commit();
        return null;
    }
}

private void validateSaleInputs(SaleDetails details)
        throws MustBeLaterThanTodayException, FileNotUploadedException {
    String etiquetas = "Etiquetas";
    if (details.getDate().before(UtilDate.trim(new Date()))) {
        throw new MustBeLaterThanTodayException(
            ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
    }
    if (details.getFile() == null) {
        throw new FileNotUploadedException(
            ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorFileNotUploadedException"));
    }
}

private void checkSaleExistence(Seller seller, String title) throws SaleAlreadyExistException {
    if (seller.doesSaleExist(title)) {
        db.getTransaction().commit();
        throw new SaleAlreadyExistException(
            ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
    }
}
```

Figura 2: Código final de create Sale

1. Keep unit interfaces small (Mantener interfaces de unidades pequeñas - Capítulo 5):

Smell detectado: El método original createSale recibía 7 parámetros (title, description, status, price, pubDate, sellerEmail, file), superando el límite máximo de 4 parámetros establecido por las guías de Building Maintainable Software (Capítulo 5).

Refactorización aplicada: Se creó la clase SaleDetails para agrupar los 6 atributos que describen conceptualmente los detalles del producto (title, desc, status, price, date, file). De este modo, los parámetros del método pasó a ser createSale(String sellerEmail, SaleDetails details), reduciendo la interfaz de 7 a solo 2 parámetros y actualizando las llamadas correspondientes en la lógica de negocio, la GUI y las pruebas unitarias.

SALEDETAILS.JAVA

```java
package domain;

import java.io.File;
import java.util.Date;

public class SaleDetails {
    private String title;
    private String desc;
    private int status;
    private float price;
    private Date date;
    private File file;

    public SaleDetails(String title, String desc, int status, float price, Date date, File file) {
        this.title = title;
        this.desc = desc;
        this.status = status;
        this.price = price;
        this.date = date;
        this.file = file;
    }

    public String getTitle() { return title; }
    public String getDesc() { return desc; }
    public int getStatus() { return status; }
    public float getPrice() { return price; }
    public Date getDate() { return date; }
    public File getFile() { return file; }
}
```

Figura 3: Código de la clase SaleDetail

Autor de esta factorización: Kailai Ye

2. Write short units of code (Escribir unidades de código cortas - Capítulo 2):

Smell detectado: El método createSale tenía más de 25 líneas de código, violando el límite de 15 líneas (LOC) (Capítulo 2) y el Principio de Responsabilidad Única (SRP), ya que mezclaba la validación de entradas, la comprobación de reglas de negocio y la gestión de transacciones en la base de datos.

Refactorización aplicada (Extract Method): Aprovechando el objeto SaleDetails, se extrajeron las validaciones previas (fecha de publicación y archivo) al método privado validateSaleInputs (SaleDetails details) y la comprobación de productos duplicados al método privado checkSaleExistence (Seller seller, String title). Como resultado, el método principal createSale quedó reducido a 14 líneas de código, logrando que todas las unidades cumplan estrictamente el límite de 15 líneas.

VALIDATESALEINPUTS

```
private void validateSaleInputs(SaleDetails details)
        throws MustBeLaterThanTodayException, FileNotUploadedException {
    String etiquetas = "Etiquetas";
    if (details.getDate().before(UtilDate.trim(new Date()))) {
        throw new MustBeLaterThanTodayException(
            ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorSaleMustBeLaterThanToday"));
    }
    if (details.getFile() == null) {
        throw new FileNotUploadedException(
            ResourceBundle.getBundle(etiquetas).getString("DataAccess.ErrorFileNotUploadedException"));
    }
}
```

Figura 4: Código del método validate SaleInputs

CHECKSALEEXISTENCE

```
private void checkSaleExistence(Seller seller, String title) throws SaleAlreadyExistException {
    if (seller.doesSaleExist(title)) {
        db.getTransaction().commit();
        throw new SaleAlreadyExistException(
            ResourceBundle.getBundle("Etiquetas").getString("DataAccess.SaleAlreadyExist"));
    }
}
```

Figura 5: Código del método checkSaleExistence

Autor de esta factorización: Kailai Ye

3. Write simple units of code (Escribir unidades de código simples - Capítulo 3):

Smell detectado: El método original createSale concentraba 4 puntos de decisión dentro de un mismo flujo de ejecución: tres estructuras if para lanzar distintas excepciones de validación (MustBeLater Than Today Exception, FileNotUploadedException y SaleAlreadyExistException) y un bloque try-catch, alcanzando una Complejidad Ciclomática de V(G)=5 Esta acumulación de caminos de ejecución alternativos dentro de la transacción dificulta la comprensión del flujo de control y el diseño de pruebas de caja blanca.

Refactorización aplicada (Decompose Conditional y Extract Method): Se desacoplaron los puntos de ramificación extrayendo las condiciones de validación de entrada al método privado validateSaleInputs (Sale Details details) y la condición de duplicidad de producto al método checkSaleExistence (Seller seller, String title).

Autor de esta factorización: Kailai Ye

" DataAccess: getProducts=> "

CÓDIGO INICIAL

DATAACCESS.JAVA (LITERAL)

```java
System.out.println(">> DataAccess: getProducts=> ");
```

Figura 6: Código inicial

CÓDIGO FINAL

DATAACCESS.JAVA (CONSTANTE)

```java
private static final String LOG_GET_PRODUCTS = ">> Data
Access: getProducts=>";
```

Figura 7: Código final

4. Write code once / Avoid Duplication (Escribir el código una sola vez - Capítulo 4):

Smell detectado: A través del análisis estático en SonarCloud, se detectó que la cadena literal ">> DataAccess: getProducts⇒ " estaba duplicada en múltiples métodos dentro de la clase DataAccess. Esto viola el principio DRY (Don't Repeat Yourself) explicado en el Capítulo 4 de Building Maintainable Software, ya que cualquier cambio en el formato de los mensajes de registro obligaría a modificar múltiples métodos manualmente, aumentando el riesgo de inconsistencias.

Refactorización aplicada (Introduce Constant / Replace Magic Literal with Symbolic Constant): Se extrajo la cadena literal repetida a una única constante estática al inicio de la clase DataAccess:

DATAACCESS.JAVA

```java
private static final String LOG_GET_PRODUCTS = ">> DataAccess: getProducts=>";
```

Figura 8: Código del variable LOG_GET_PRODUCTOS

Posteriormente, se reemplazaron todas las apariciones del literal en los métodos afectados por la referencia a LOG_GET_PRODUCTS.

Autor de esta factorización: Kailai Ye

addSale

CÓDIGO INICIAL

SELLER.JAVA (INICIAL)

```java
public Sale addSale(String title, String description,
                    int status, float price,
                    Date pubDate, File file) {
    Sale sale = new Sale(title, description,
                         status, price, pubDate,
                         file, this);
    sales.add(sale);
    return sale;
}
```

Figura 9: Código inicial de addSale

CÓDIGO FINAL

SELLER.JAVA (REFACTORIZADO)

```java
public Sale addSale(SaleDetails details) {
    Sale sale = new Sale(details.getTitle(),
                         details.getDesc(),
                         details.getStatus(),
                         details.getPrice(),
                         details.getDate(),
                         details.getFile(), this);
    sales.add(sale);
    return sale;
}
```

Figura 10: Código final de addSale

1. Keep unit interfaces small (Mantener interfaces de unidades pequeñas - Capítulo 5):

Smell detectado: En la clase del dominio domain. Seller, el método addSale(String title, String description, int status, float price, Date pubDate, File file) recibía 6 parámetros, superando el límite máximo de 4 parámetros por unidad establecido en las directrices de Building Maintainable Software (Capítulo 5). Además, tras haber introducido el objeto SaleDetails en DataAccess.createSale, se producía un acoplamiento innecesario al tener que descomponer nuevamente el objeto details mediante seis llamadas sucesivas a sus métodos getter (details.getTitle(), details.getDesc(), etc.).

Refactorización aplicada (Introduce Parameter Object): Se reutilizó la clase SaleDetails para propagar el objeto de parámetros hasta la capa de dominio. Se modificó la firma del método en la clase Seller a public Sale addSale(SaleDetails details), encapsulando la extracción de los atributos dentro de dicho método al instanciar el objeto Sale. De este modo, la llamada desde DataAccess.createSale se simplificó directamente a Sale sale = seller.addSale(details).

Autor de esta factorización: Rafael Concejo

aceptarOferta

CÓDIGO INICIAL

DATAACCESS.JAVA (INICIAL)

```java
public void aceptarOferta(Offer o) {
    db.getTransaction().begin();
    Sale v = db.find(Sale.class, o.getSale().getId());
    User comprador = db.find(User.class, o.getBuyer().g
etEmail());
    User vendedor = db.find(User.class, v.getSeller().g
etEmail());

    float precioFinal = o.getPrice();
    float a = comprador.getSaldo();
    comprador.setSaldo(comprador.getSaldo() - precioFin
al);
    comprador.addTransaccion(precioFinal, "COBRO", "Pag
o por: " + v.getTitle(), a, a - precioFinal);
    db.persist(comprador);

    float b = vendedor.getSaldo();
    vendedor.setSaldo(vendedor.getSaldo() + precioFina
l);
    vendedor.addTransaccion(precioFinal, "INGRESO", "Ve
nta de: " + v.getTitle(), b, b + precioFinal);
    db.persist(vendedor);

    for (Offer of : v.getOfertas()) {
        if (of.equals(o)) {
            of.setAccepted(1);
        } else {
            of.setAccepted(-1);
        }
    }
    v.setVendido(true);
    db.persist(v);
    db.getTransaction().commit();
}
```

Figura 11: Código inicial de aceptarOferta

CÓDIGO FINAL

DATAACCESS.JAVA (REFACTORIZADO)

```java
public void aceptarOferta(Offer o) {
    db.getTransaction().begin();
    Sale v = db.find(Sale.class, o.getSale().getId());
    User comprador = db.find(User.class, o.getBuyer().g
etEmail());
    User vendedor = db.find(User.class, v.getSeller().g
etEmail());

    procesarTransaccionEconomica(comprador, vendedor,
o.getPrice(), v.getTitle());
    actualizarEstadoVentaYOfertas(v, o);
    db.getTransaction().commit();
}

private void procesarTransaccionEconomica(User comprado
r, User vendedor,
                                          float precio
Final, String titulo) {
    float saldoComprador = comprador.getSaldo();
    comprador.setSaldo(saldoComprador - precioFinal);
    comprador.addTransaccion(precioFinal, "COBRO", "Pag
o por: " + titulo,
                             saldoComprador, saldoCompr
ador - precioFinal);
    db.persist(comprador);

    float saldoVendedor = vendedor.getSaldo();
    vendedor.setSaldo(saldoVendedor + precioFinal);
    vendedor.addTransaccion(precioFinal, "INGRESO", "Ve
nta de: " + titulo,
                            saldoVendedor, saldoVendedo
r + precioFinal);
    db.persist(vendedor);
}

private void actualizarEstadoVentaYOfertas(Sale v, Offe
r o) {
    for (Offer of : v.getOfertas()) {
        if (of.equals(o)) {
            of.setAccepted(1);
        } else {
            of.setAccepted(-1);
        }
    }
    v.setVendido(true);
    db.persist(v);
}
```

Figura 12: Código refactorizado y métodos extraídos de aceptarOferta

2. Write short units of code (Escribir unidades de código cortas - Capítulo 2):

Smell detectado: El método aceptarOferta (Offer o) de la clase DataAccess contaba con 25 líneas de código efectivas, superando el límite máximo de 15 líneas por unidad establecido en el Capítulo 2 de Building Maintainable Software. Asimismo, el método incumplía el Principio de Responsabilidad Única (SRP) al concentrar múltiples responsabilidades en un solo bloque monolítico: la gestión de la transacción y recuperación de entidades desde la base de datos, el cálculo de saldos y registro de transacciones financieras ("COBRO" e "INGRESO") tanto para el comprador como para el vendedor, y la iteración sobre todas las ofertas del producto para actualizar sus estados (1 o-1) y marcar la venta como finalizada (setVendido(true)).

Refactorización aplicada (Extract Method): Se aplicó la técnica Extract Method para descomponer el método según sus responsabilidades lógicas en dos nuevos métodos auxiliares:

procesar Transaccion Economica (User comprador, User vendedor, float precioFinal, String titulo): encapsula exclusivamente la actualización de saldos, la creación de las transacciones económicas y la persistencia de ambos usuarios.

PROCESARTRANSACCIONECONOMICA

```
private void procesarTransaccionEconomica(User comprador, User vendedor,
                                          float precioFinal, String titulo) {
    float saldoComprador = comprador.getSaldo();
    comprador.setSaldo(saldoComprador - precioFinal);
    comprador.addTransaccion(precioFinal, "COBRO", "Pago por: " + titulo,
                             saldoComprador, saldoComprador - precioFinal);
    db.persist(comprador);

    float saldoVendedor = vendedor.getSaldo();
    vendedor.setSaldo(saldoVendedor + precioFinal);
    vendedor.addTransaccion(precioFinal, "INGRESO", "Venta de: " + titulo,
                            saldoVendedor, saldoVendedor + precioFinal);
    db.persist(vendedor);
}
```

Figura 13: Código del método procesar Transaccion Economica

actualizarEstado Venta YOfertas (Sale v, Offer o): encapsula el bucle for que actualiza el estado de las ofertas asociadas al producto y marca la venta como vendida.

ACTUALIZARESTADOVENTAYOFERTAS

```
private void actualizarEstadoVentaYOfertas(Sale v, Offer o) {
    for (Offer of : v.getOfertas()) {
        if (of.equals(o)) {
            of.setAccepted(1);
        } else {
            of.setAccepted(-1);
        }
    }
    v.setVendido(true);
    db.persist(v);
}
```

Figura 14: Código del método actualizarEstado Venta YOfertas

Autor de esta factorización: Rafael Concejo

getProductos Comprados

CÓDIGO INICIAL

DATAACCESS.JAVA (INICIAL)

```
public List<Offer> getProductosComprados(Buyer bu, Stri
ng desc) {
    if (bu == null || desc == null) {
        return new ArrayList<Offer>();
    }
    System.out.println(">> DataAccess: getProducts=>
");
    List<Offer> res = new ArrayList<Offer>();
    TypedQuery<Offer> query = db.createQuery(
        "SELECT o FROM Offer o JOIN o.sale s WHERE s.ti
tle LIKE ?1", Offer.class);
    query.setParameter(1, "%" + desc + "%");

    List<Offer> offers = query.getResultList();
    for (Offer offer : offers) {
        if (offer.getBuyer().getEmail().equals(bu.getEm
ail())) {
            if (offer.getAccepted() == 1) {
                res.add(offer);
            }
        }
    }
    return res;
}
```

Figura 15: Código inicial de getProductos Comprados

CÓDIGO FINAL

DATAACCESS.JAVA (REFACTORIZADO)

```
public List<Offer> getProductosComprados(Buyer bu, Stri
ng desc) {
    if (bu == null || desc == null) {
        return new ArrayList<Offer>();
    }
    System.out.println(">> DataAccess: getProducts=>
");
    List<Offer> res = new ArrayList<Offer>();
    TypedQuery<Offer> query = db.createQuery(
        "SELECT o FROM Offer o JOIN o.sale s WHERE s.ti
tle LIKE ?1", Offer.class);
    query.setParameter(1, "%" + desc + "%");

    List<Offer> offers = query.getResultList();
    for (Offer offer : offers) {
        if (isOfertaAceptadaDelComprador(offer, bu)) {
            res.add(offer);
        }
    }
    return res;
}

private boolean isOfertaAceptadaDelComprador(Offer offe
r, Buyer bu) {
    return offer.getBuyer().getEmail().equals(bu.getEma
il()) && offer.getAccepted() == 1;
}
```

Figura 16: Código final de getProductos Comprados

3. Write simple units of code (Escribir unidades de código simples - Capítulo 3):

Smell detectado: El método original getProductos Comprados (Buyer bu, String desc) presentaba una alta complejidad ciclomática y un anidamiento excesivo. Contaba con 5 puntos de condiciones (if, operador ||, bucle for y dos bloques if anidados), lo que resultaba en una Complejidad Ciclomática de 6, superando el límite máximo de 4 puntos de ramificación recomendado en el Capítulo 3 de Building Maintainable Software. Además, contenía 3 niveles de anidamiento (for + if + if), dificultando la lectura y las pruebas del flujo de control.

Refactorización aplicada (Extract Method y Consolidate Conditional Expression): Se unificaron las dos condiciones if anidadas dentro del bucle for y se extrajeron a un nuevo método booleano llamado is OfertaAceptadaDelComprador (Offer offer, Buyer bu). Tras la refactorización, se eliminó el anidamiento del bucle inicial. Las condiciones del método getProductos Comprados se redujeron a 4 (Complejidad Ciclomática = 5) y el nuevo método auxiliar isOfertaAceptadaDelComprador tiene solo una condición (Complejidad Ciclomática = 2), cumpliendo estrictamente con las métricas de simplicidad.

ISOFERTAACEPTADADELCOMPRADOR

```
private boolean isOfertaAceptadaDelComprador(Offer offer, Buyer bu) {
    return offer.getBuyer().getEmail().equals(bu.getEmail()) && offer.getAccepted() == 1;
}
```

Figura 17: Código del método is OfertaAceptadaDelComprador

Autor de esta factorización: Rafael Concejo

"SELECT FROM Offer o JOIN o.sale s WHERE s.title LIKE ?1"

CÓDIGO INICIAL

JPQL LITERAL DUPLICADO

```
"SELECT o FROM Offer o JOIN o.sale s WHERE s.title LIKE
?1"
```

Figura 18: Código Inicial

CÓDIGO FINAL

CONSTANTE QUERY_OFFER

```
private static final String QUERY_OFFER =
    "SELECT o FROM Offer o JOIN o.sale s WHERE s.title
LIKE ?1";
```

Figura 19: Código final

4. Write code once / Avoid Duplication (Escribir el código una sola vez - Capítulo 4):

Smell detectado: Mediante la inspección del código y el análisis en SonarCloud, se detectó que la consulta "SELECT FROM Offer o JOIN o.sale s WHERE s.title LIKE ?1" se encontraba codificada de forma rígida y duplicada en múltiples métodos de consulta dentro de la clase DataAccess. Esta repetición vulnera el principio DRY expuesto en el Capítulo 4 del libro guía, ya que cualquier cambio futuro en el esquema o en los nombres de los atributos de las entidades Offer o Sale (como o.sale o s.title) obligaría a buscar y modificar manualmente la misma cadena en varios lugares, incrementando el riesgo de errores en tiempo de ejecución.

Refactorización aplicada (Introducir Constante): Se extrajo la cadena de la consulta JPQL a una única constante privada y estática al inicio de la clase DataAccess. Posteriormente, se sustituyeron todas las apariciones del duplicado en las llamadas a db.createQuery(...) por la referencia a la constante QUERY_OFFER.

Autor de esta factorización: Rafael Concejo

Referencia e Información

Con las refactorizaciones aplicadas a lo largo de este trabajo sobre la clase DataAccess y el modelo de dominio, el código cumple ahora rigurosamente con las directrices de mantenibilidad en cuanto a interfaces pequeñas, unidades cortas, simplicidad estructural y ausencia de código duplicado. (En escalas pequeñas)

En relación con el principio de escribir unidades de código simples (Capítulo 3), cabe señalar que, además del caso evidente de anidamiento corregido en getProductos Comprados, dentro del proyecto únicamente el método createSale presentaba un problema de complejidad menos evidente (situándose justo en el límite de complejidad ciclomática de 5 debido a sus comprobaciones previas y al bloque de captura de excepciones), el cual quedó resuelto simultáneamente al extraer sus métodos privados de validación.

Por último, para asegurar que la batería de pruebas siguiera siendo totalmente funcional y compatible tras las modificaciones realizadas en el código, se llevaron a cabo las adaptaciones pertinentes en las clases de prueba, logrando que todas las pruebas de caja negra y caja blanca se ejecuten con éxito y verificando que el comportamiento externo del sistema se mantiene intacto.

Referencia:

Visser, J., Rigal, S., van der Leek, R., van Eck, P., & Wijnhoven, G. (2016). Building maintainable software, Java edition: Ten guidelines for future-proof code. O'Reilly Media.
https://www.labunix.uqam.ca/~tremblay_gu/MGL7460/Liens/Building_Maintainable_Software_SIG.pdf

Iturrioz, J. y Goñi, A. (s.f.). Ingeniería del Software II. Tema 2: Mantenimiento Software [Diapositivas de curso].
https://egela.ehu.eus/pluginfile.php/12640633/mod_resource/content/9/Mantenimiento25-c.pdf

Apartado uso de IA

Declaración sobre el uso de herramientas de Inteligencia Artificial:

Se deja constancia explícita de que las herramientas de inteligencia artificial se han utilizado **única y exclusivamente para la maquetación visual, el formato tipográfico y el rediseño estético del documento** con el fin de optimizar su presentación y exportación a PDF.

En ningún momento se ha empleado la inteligencia artificial para generar, deducir o falsear datos, métricas, resultados ni fragmentos lógicos del proyecto. Todos los datos analizados, los olores de código identificados (*code smells*), las soluciones de refactorización implementadas y la autoría de cada cambio corresponden íntegra y fielmente al trabajo técnico real realizado por los autores.